

Homework 18: A Counter That Cannot Count

Build the same counter four ways, watch one of them fail, then fix a race that a thread-safe collection does **not** protect you from.

Your program must exit cleanly every time. Any thread that might not finish has to be a daemon, and any `join` has to have a timeout.

Part One: Four Counters

Create `BankAccount.java` in the classic form.

Define a `Counter` interface with `increment()`, `value()` and `name()`, then four implementations:

Name	Mechanism
<code>UnsafeCounter</code>	a plain <code>long count</code> and <code>count++</code>
<code>SynchronizedCounter</code>	a <code>synchronized</code> block
<code>AtomicCounter</code>	<code>AtomicLong</code>
<code>AdderCounter</code>	<code>LongAdder</code>

Run each through the same workload: 4 threads, 100,000 increments each. Print the result, whether it is correct, and how long it took.

One requirement on the synchronized version: lock on a `private final Object`, not on `this`.

Write a comment saying what could go wrong with `this`, and name two other things that are also bad to lock on and why.

Explain It

A comment covering, in your own words:

- What `count++` actually compiles to
- A concrete interleaving of two threads that loses exactly one increment
- Why the number lost **differs every run**, and why that is worse for you than a consistently wrong answer

Part Two: A Loop That Never Ends

Write a class holding two fields: a plain `boolean` and a `volatile boolean`.

For each one:

1. Start a reader thread that spins until the flag is true
2. Sleep briefly, set the flag from `main`
3. `join` with a timeout, then print whether the reader is still alive

The plain one will still be alive. The volatile one will not.

Explain It

This is the part worth getting right. Write a comment covering:

- **Why this is not a race.** How many threads write, and how many times?
- What the compiler is permitted to do to that loop, and why it is permitted
- What `volatile` changes
- **Why `volatile` would not fix part one's counter.** Name the two separate problems and say which one `volatile` solves

Part Three: Check Then Act

Use a `ConcurrentHashMap`. Have 8 threads all try to claim the same key:

```
if (!map.containsKey("winner")) {
    map.put("winner", id);
    winners.add(id);
}
```

Print how many threads believed they won. It will be more than one.

Then fix it so exactly one can win, and print that too.

The Question That Matters

The map is a `ConcurrentHashMap`, which is thread-safe. Write a comment explaining why that did **not** prevent the race, what a thread-safe collection does and does not guarantee, and why `putIfAbsent`, `compute` and `merge` exist at all if the map is already safe.

Hint on making the race actually fire: if each thread starts and finishes before the next begins, nothing collides. Two `CountDownLatch` objects will let you hold every thread at the gate and release them together.

Part Four: The Failure Nobody Hears

Submit a task to an `ExecutorService` that throws, and **never inspect the Future**. Show that nothing at all is reported.

Then submit the same task and call `get()`, catching what comes back. Print the cause.

List three ways to make sure a failure is never lost.

Acceptance Criteria

- ☐ Runs with `java BankAccount.java` and exits cleanly every time
- ☐ All four counters run the identical workload with timings
- ☐ The unsafe one is visibly wrong; the other three are exactly correct
- ☐ `synchronized` locks on a private final object, with a comment on why
- ☐ Two other bad lock targets are named with reasons
- ☐ The explanation gives a concrete losing interleaving
- ☐ The plain flag reader is still alive after the join; the volatile one is not
- ☐ Reader threads are daemons and joins have timeouts
- ☐ The explanation says why this is not a race and why `volatile` is not enough for part one
- ☐ The broken check-then-act reports more than one winner
- ☐ The fixed version reports exactly one
- ☐ A comment explains what `ConcurrentHashMap` does and does not guarantee
- ☐ The silent failure is demonstrated and then surfaced

Stretch

Replace the fixed check-then-act with `compute`, so the winning thread also gets to record a timestamp atomically alongside its id.

Then write a comment on what `compute` holds while your lambda runs, and why calling another map's method, doing IO, or acquiring a second lock inside that lambda would be a bad idea.

Hint, if the unsafe counter comes out correct

Check that the threads genuinely overlap. If your workload is small enough, or the pool is size 1, each task finishes before the next starts and there is nothing to interleave. More iterations and at least as many threads as cores will do it.